



Introduzione ad Android

Sistemi Operativi

Davide Loconte

Ingegneria Informatica e dell'Automazione
(A.A. 2024/2025)

- Introduzione
- Architettura Android
 - Linux Kernel
 - Hardware Abstraction Layer
 - Core libraries
 - Android Runtime
 - Application Framework
 - Applications
- Android Startup
- Interazione tra i layer

Dispositivi mobile



↳ SMARTPHONE, TABLET, SMARTWATCH...

Un **dispositivo mobile** è un calcolatore elettronico **leggero** e **portatile**. Il **sistema operativo** deve tenere conto delle **caratteristiche** di questi dispositivi:

- **Risorse di calcolo** limitate
- **Input e output:** sistemi di interfacciamento eterogenei e vincolati dalle dimensioni dei dispositivi
- **Affidabilità delle funzionalità base**, ad esempio garantendo la connettività telefonica
- **Energia limitata:** Ottimizzazione del dispendio energetico per prolungare la durata delle batterie
- **Eterogeneità:** Supporto a diverse configurazioni hardware

Introduzione ad Android



AOSP – Android Open Source Project

- **Android** è un sistema operativo **open source** per **dispositivi mobili**.
- **Google inc.** è il leader dell'Android Open Source Project
- Il target di riferimento di Android sono i dispositivi portatili consumer, come tablet, smartphone e wearables
- L'obiettivo è fornire una piattaforma di riferimento comune a tutti i dispositivi portatili (*MODIFICARE AL CODICE MINIMALI*)
- Android è un progetto **open source**
 - Il codice e la documentazione sono pubblici e disponibili a tutti
 - Il kernel linux è rilasciato sotto **GNU General Public Licence** (GPL)
 - La piattaforma Android è rilasciata sotto **Open Source Apache Software Licence 2.0** (ASL)

Caratteristiche di Android



RECAP CARATTERISTICHE ANDROID:

Basato su Linux: Utilizza il kernel Linux per gestire le risorse hardware

Multitasking e Gestione delle Risorse: Supporta l'esecuzione simultanea di più applicazioni e gestisce le risorse in modo efficiente.

Interfaccia Utente Intuitiva: Offre un'interfaccia utente facile da utilizzare e comprendere e allineata con le caratteristiche dei dispositivi di riferimento (TV, Smartphone, Smartwatch, Tablet, Auto ...)

Sicurezza e Virtualizzazione: Utilizza la virtualizzazione per isolare le applicazioni e garantire la sicurezza del sistema.

Gestione dell'Energia: Implementa strategie per ottimizzare il consumo energetico e prolungare la durata della batteria.

Framework di Sviluppo: Fornisce strumenti per lo sviluppo di applicazioni native.

Ecosistema di Applicazioni: Accesso al Google Play Store per scaricare applicazioni e giochi

2003: Andy Rubin, Rich Miner, Nick Sears e Chris White fondano la **Android Inc.**

2005: Android Inc. viene **acquisito da Google**

2007: Viene fondata la **Open Handset Alliance**, consorzio comprendente 84 (2015) membri tra cui produttori di hardware, di software e compagnie di telecomunicazione (oltre a Google stessa), con lo scopo di realizzare congiuntamente tutto il necessario per la diffusione del sistema → standard aperti in ambito mobile

2007: viene rilasciato con licenza **Apache una prima versione Android Beta SDK**

2008: Android 1.0 (eseguibile solo su HTC G1)

2009: Android 1.5 Cupcake: **prima versione commerciale** installata su più dispositivi

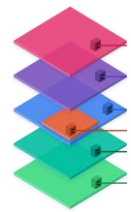
2011-2012: Android 4.0 Ice Cream Sandwich **unifica** l'esperienza utente tra smartphone e tablet

2014: Android 4.4W KitKat per smartwatch e Android 5.0 Lollipop con interfaccia **Material Design**

2016: Android 7.0 Nougat introduce **ART (Android Runtime)** come runtime predefinito al posto di Dalvik, migliorando le prestazioni e l'efficienza del sistema

2017+: Continui aggiornamenti con Android 8.0 Oreo, Android 9.0 Pie e Android 10, che introducono nuove **funzionalità** e **ottimizzazioni**

Architettura di Android



ARCHITETTURA A LIVELLI :

Apps

Alarm - Browser - Calculator - Calendar - Camera - Clock - Contacts - IM
Dialer - Email - Home - Media Player - Photo Album - SMS/MMS - Voice Dial

Framework

Content Providers - Activity Manager - Location Manager - View System
Package Manager - NotificationManager - Resource Manager
Telephony Manager - Window Manager

Native Libraries

Audio Manager - LIBC - SSL - Freetype - Media
OpenGL/ES - SQLite - Webkit - Surface Manager

Runtime

Core Libraries
Android Runtime (ART)

HAL

Audio - Bluetooth - Camera - DRM - External Storage - Graphics - Input
Media - Sensors - TV

Linux Kernel

Drivers (Audio - Binder (IPC) - Bluetooth
Camera Display - Keypad Shared Memory
USB Wi-fi) - Power Management

Secure Element

Trusted Execution Environment

Linux
Kernel

Drivers (Audio - Binder (IPC) - Bluetooth
Camera Display - Keypad Shared Memory
USB Wi-fi) - Power Management

Secure
Element

Trusted Execution
Environment

↳ COMUNICAZIONE CON L'HARDWARE E GESTIONE DRIVER



Android adopera un **kernel** Linux con alcune modifiche:

- Utilizza un **Low Memory Killer** personalizzato
- Power management basato sui **wakelocks**
- Implementa il **Binder IPC driver**, un sistema per la comunicazione tra processi
- Servizio di logging centralizzato (**logcat**)

NUOVE VERSIONI
MODIFICATE LA GESTIONE
DELLA APP PER CAMBIARE!

Il **Secure Element** è una componente hardware che memorizza informazioni sensibili, come dati per i pagamenti con carte elettroniche e NFC

Linux e Android condividono lo stesso workflow nello sviluppo di driver e moduli kernel

Low Memory Killer (LMK): Gestione memoria aggressiva basata sul concetto di pressione della memoria (**vmpressure**)

- Il demone **lmkd** misura i ritardi nell'esecuzione dei task dovuti alla mancanza di memoria, più ritardi indicano una pressione maggiore
- la **vmpressure** è utilizzata per scegliere quali task killare e quando per alleviare la pressione

Gestione dell'energia basata sui wakelocks

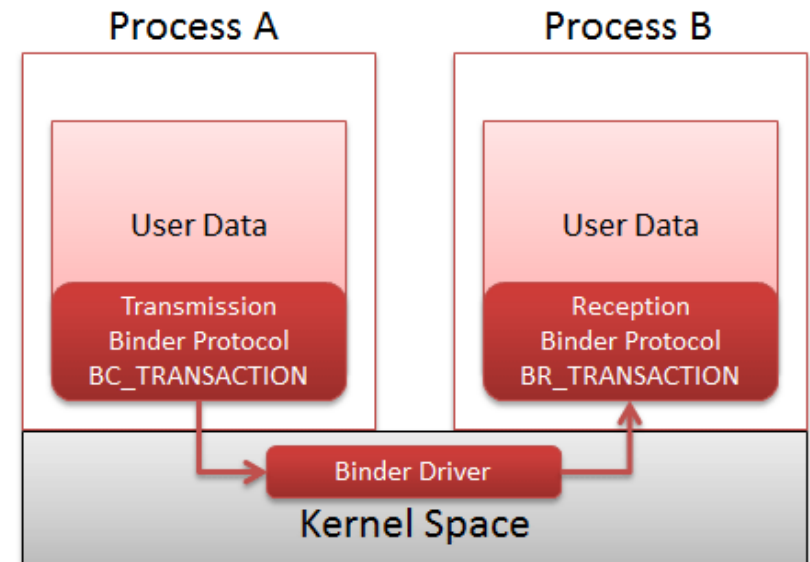
- Le app devono richiedere esplicitamente un wakelock per poter tenere attiva una risorsa
- In assenza di wakelocks, Android disattiva le risorse per risparmiare energia
- Lavora insieme ai servizi di gestione energia di Linux

Flag Value	CPU	Screen	Keyboard
PARTIAL_WAKE_LOCK	On*	Off	Off
SCREEN_DIM_WAKE_LOCK	On	Dim	Off
SCREEN_BRIGHT_WAKE_LOCK	On	Bright	Off
FULL_WAKE_LOCK	On	Bright	Bright

Binder IPC Driver -> OpenBinder

(Consumo ridotto di memoria)

- Sistema di Inter Process Communication **sincrona** usato da Android
- È un **canale di comunicazione comune** che consente di condividere oggetti tra processi
- Il driver gestisce autonomamente Allocazione e deallocazione degli oggetti tramite **Reference Counting**
- A ciascun processo è associato un **pool di threads** per la gestione della comunicazione



I dispositivi di **storage** utilizzati nei device portatili sono **memorie flash** (flash memory)

- **No seek time** -> una memoria flash non ha alcuna **latenza** causata da un'operazione di ricerca, maggiori prestazioni negli accessi casuali
- **Block erasing** -> nelle flash memory i blocchi devono essere **esplicitamente cancellati** prima che vi sia nuovamente consentita la scrittura. Questa è una operazione che richiede tempo e pertanto deve essere eseguita negli istanti idle
- **Wear leveling** -> il numero di scritture su una singola cella è limitato, occorre **distribuire le letture e le scritture** sulla memoria per evitare di deteriorare eccessivamente una singola area di memoria

I requisiti di un file system per dispositivi mobili sono:

- **Robusto** -> supporto al Journaling, deve gestire il wear leveling e i bad blocks
- **Garbage collection** -> cancella e compatta i blocchi quando il dispositivo è idle
- **Portabile** -> deve essere compatibile con diversi tipi di dispositivi

Android eredita da Linux sia il **directory tree** (il file system è **Posix compliant**) che i **file systems**

- ext4, f2fs, YAFFS2 ...

Hardware Abstraction Layer

HAL

HAL

Audio - Bluetooth - Camera - DRM - External Storage - Graphics - Input
Media - Sensors - TV

- Un **HAL** definisce una interfaccia standard tra driver di basso livello e i servizi di alto livello
- Consente di implementare funzionalità senza modificare i livelli più alti
- In fase di sviluppo, il vendor deve definire una HAL usando il linguaggio **HAL Interface Definition Language – HIDL**
- Grazie ad HAL, Android è **agnostico** rispetto all'hardware ed è più semplice aggiornare la versione di Android

Native Libraries

Audio Manager - LIBC - SSL - Freetype - Media
OpenGL/ES - SQLite - Webkit - Surface Manager

- Funzionalità di basso livello -> core del Sistema
 - grafica, rendering di pagine web o decoding/encoding di dati multimediali)
- **BioniC** -> implementazione della libreria standard C derivate da BSD. Occuma meno storage di GNU C Library e **ottimizzata** per dispositivi embedded
- L'accesso alle funzionalità native non è diretto, ma avviene tramite l'application Framework

- **SQLite**: DBMS relazionale leggero e potente, il formato del file è indipendente dalla piattaforma
- **Surface Manager**: Libreria per la gestione dell'interfaccia grafica
- **SGL e OpenGL/ES**: gestiscono rispettivamente la grafica 2D e 3D
- **Media frameworks**: basato su Open Core e Packet Video, fornisce funzioni e codec per gestire la riproduzione di contenuti multimediali
- **FreeType**: strumento scritto in C per rasterizzare i caratteri
- **WebKit/Blink**: Rendering engine per le pagine web usato da Opera e Chrome

Runtime

Core Libraries
Android Runtime (ART)

- **Run-time system** per le applicazioni Android
- Esegue i file in formato *.dex* e *.oat*
- Macchina virtuale **register-based**
 - Rispetto alle macchine stack-based (ad esempio **JVM**), il codice occupa più memoria, ma complessivamente vengono eseguite meno istruzioni
- Le applicazioni sono distribuite in un pacchetto in formato *.apk* e contiene il codice *.dex* + risorse utili per l'app

.dex -> Dalvik EXecutable

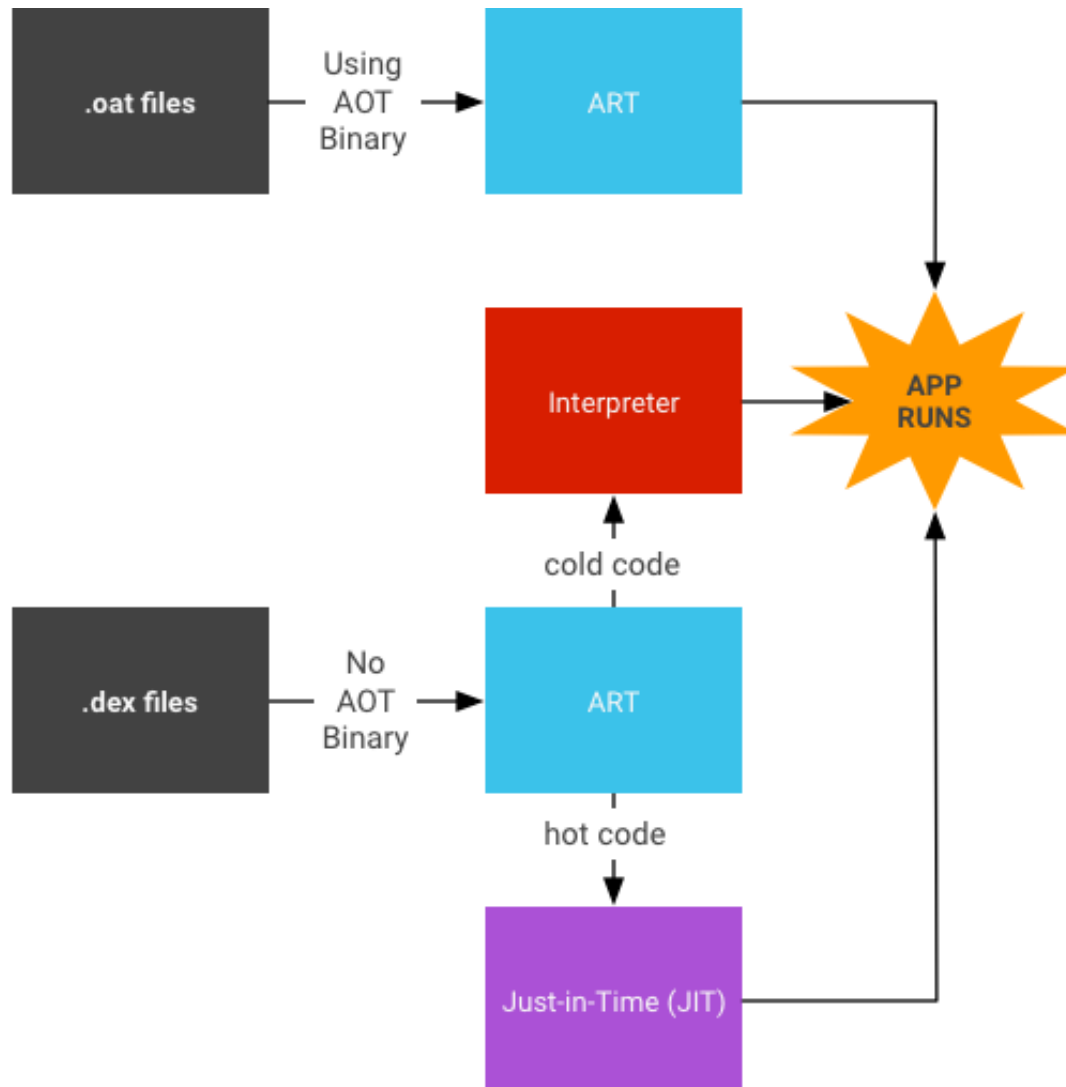
- Codice interpretato indipendente dalla macchina
 - Favorisce la portabilità a discapito delle performance
- Disponibile subito dopo l'installazione dell'applicazione
- La parte di codice più utilizzata (**Hot-Code**) viene compilata Just-In-Time, il resto (**Cold-Code**) è puramente interpretato

.oat -> codice binario

- Codice binario direttamente eseguibile
- Generato dal compilatore **dex2oat**
 - La compilazione avviene automaticamente in maniera trasparente all'utente quando il dispositivo in ricarica e in idle.
 - I tempi di esecuzione sono minori, ma è necessario memorizzare anche i binari -> maggiore consumo di storage
- Compilazione Ahead of Time -> AOT

Android Runtime 3/3

Runtime



Framework

Content Providers - Activity Manager - Location Manager - View System
Package Manager - NotificationManager - Resource Manager
Telephony Manager - Window Manager

- L'**Application Framework** è l'insieme di **librerie** e di **software** che forniscono l'**ambiente** e i **servizi** necessari per l'esecuzione delle applicazioni sviluppate con l'SDK Android
- Le API di alto livello chiamano le funzioni native per eseguire le operazioni

- **Resource Manager**: garantisce l'accesso a **risorse** che non rappresentano codice (stringhe, grafici, immagini, file di layout e configurazione)
- **Notification Manager**: permette a tutte le applicazioni di gestire **alert personalizzati** verso l'utente sotto forma di vari eventi (icona nella barra di notifica, accensione LED, suoni o vibrazioni)
- **Activity Manager**: le Activity sono entità associate ad una **schermata** dell'applicazione. Gestisce il **ciclo di vita** delle **activity** e fornisce un meccanismo elementare per la navigazione tra le varie schermate
- **Telephony Manager**: gestisce l'interazione con le funzionalità caratteristiche di un **telefono** (avviare una telefonata, controllarne lo stato, SMS)

- **Location Manager**: consente alle Location Based Application (LBA) di utilizzare API per accedere a funzioni legate alla **location**, tra cui le operazioni di **georeferenziazione** (e attivare eventi date delle precondizioni)
- **Window Manager**: permette di **gestire le finestre** delle diverse applicazioni, gestite da processi diversi, sullo schermo del dispositivo. Astrazione con API Java, dei servizi nativi del Surface Manager
- **Package Manager**: gestisce i processi di **installazione** e **rimozione** delle applicazioni dal sistema
- **Content Providers**: permettono alle applicazioni di **accedere ai dati** di altre applicazioni (es. la rubrica) o di mettere in condivisione i loro stessi dati
- **View System**: gestisce l'insieme **degli elementi grafici** utilizzati **nella costruzione dell'interfaccia** verso l'utente (bottoni, griglie, text boxes)

Alarm - Browser - Calculator - Calendar - Camera - Clock - Contacts - IM
Dialer - Email - Home - Media Player - Photo Album - SMS/MMS - Voice Dial

I principali linguaggi per lo sviluppo di **Applicazioni** sono **Kotlin** e **Java**

- L'applicazione compilata è impacchettata attraverso il tool **aapt** in un package Android **.apk** che è possibile installare sui dispositivi Android

Caratteristiche delle applicazioni Android:

- Ogni applicazione è un singolo processo Linux, ha la propria istanza di ART ed ha assegnato un unico Linux user ID
- Ogni applicazione può pubblicare le sue capabilities e le altre possono farne uso
- Le applicazioni Android non hanno un singolo entry point



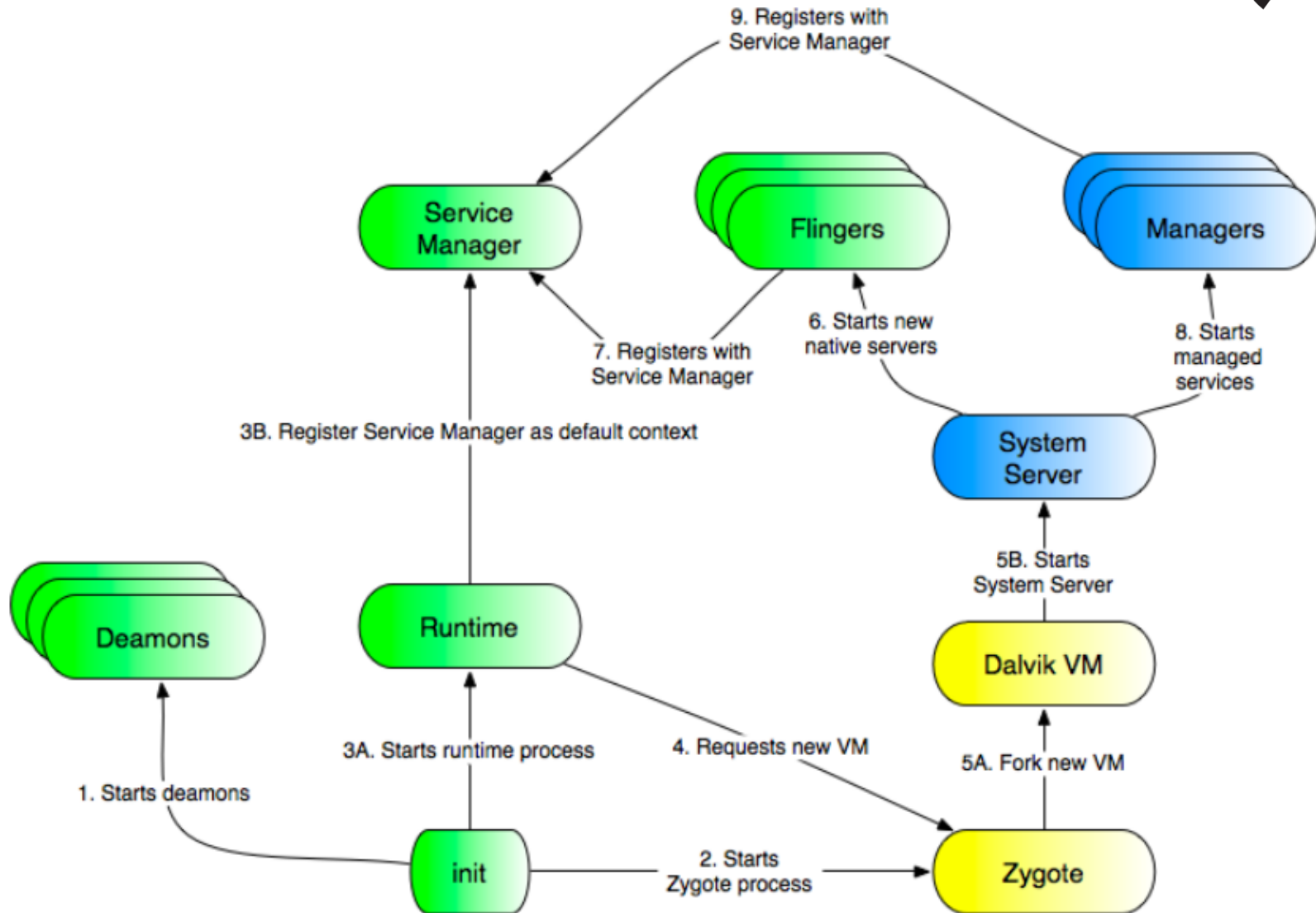
All'avvio, il kernel invoca lo script **init** che inizializza:

- I demoni di sistema (lkmd, usbd, rild...)
- Il runtime process -> avvia il service manager che gestisce la registrazione dei servizi e il lookup
- Il servizio Zygote

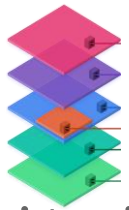
Zygote è il processo padre delle istanze di ART nelle quali girano le applicazioni

- Zygote rimane in ascolto sul proprio socket, quando arriva una richiesta viene forkato inizializzando una nuova istanza di ART
- Precarica tutte le librerie java condivise riducendo drasticamente i tempi di avvio
- copy-on-write() -> le risorse sono duplicate solo dopo una scrittura
- Avvia il System Server per la gestione dei servizi base del sistema Android

Start-up 2/2



Interazione tra livelli 1/4



Le API di **alto livello** eseguono chiamate a **funzioni native**. Le interazioni tra i livelli possono avere differenti forme:

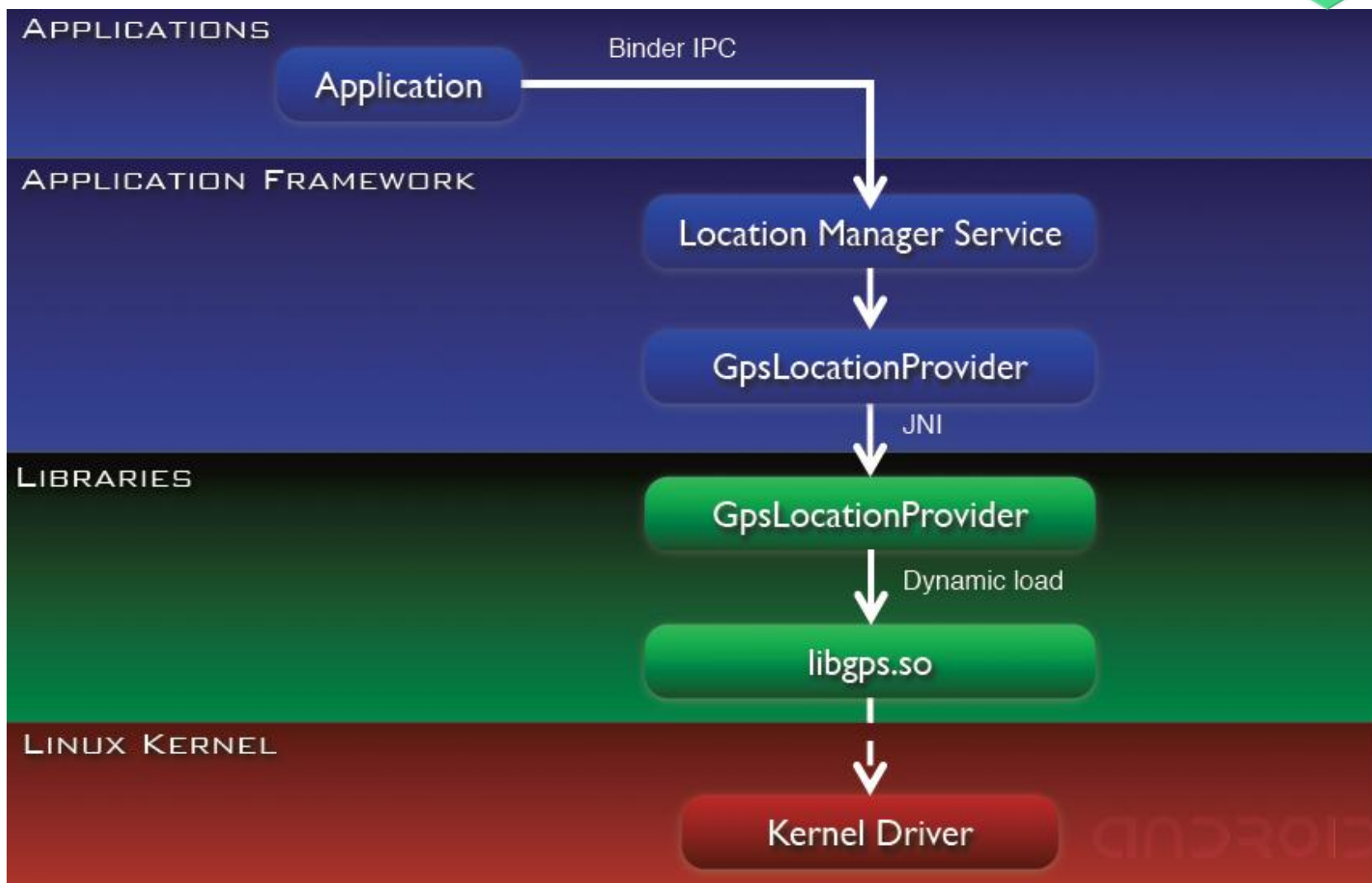
- App -> Runtime Service -> lib
- App -> Runtime Service -> Native Service -> lib
- App -> Runtime Service -> Native Daemon -> lib

Le chiamate tra framework applicativi e librerie native avviene tramite **Java Native Interface**, le componenti native comunicano con **Binder IPC** o tramite **socket**.

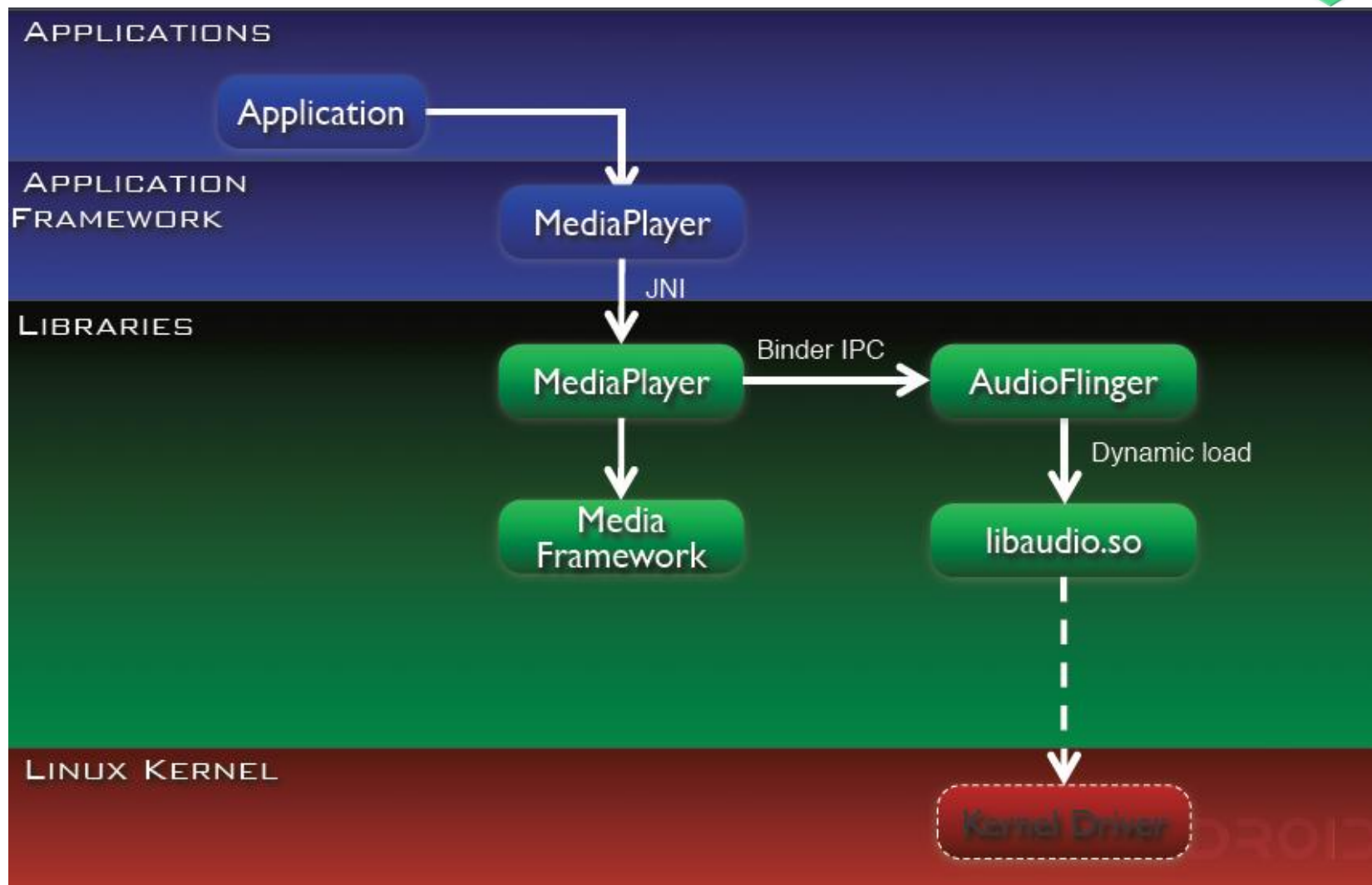
○ Esempi

- Servizi di geolocalizzazione
- Media Player
- Servizio di telefonia

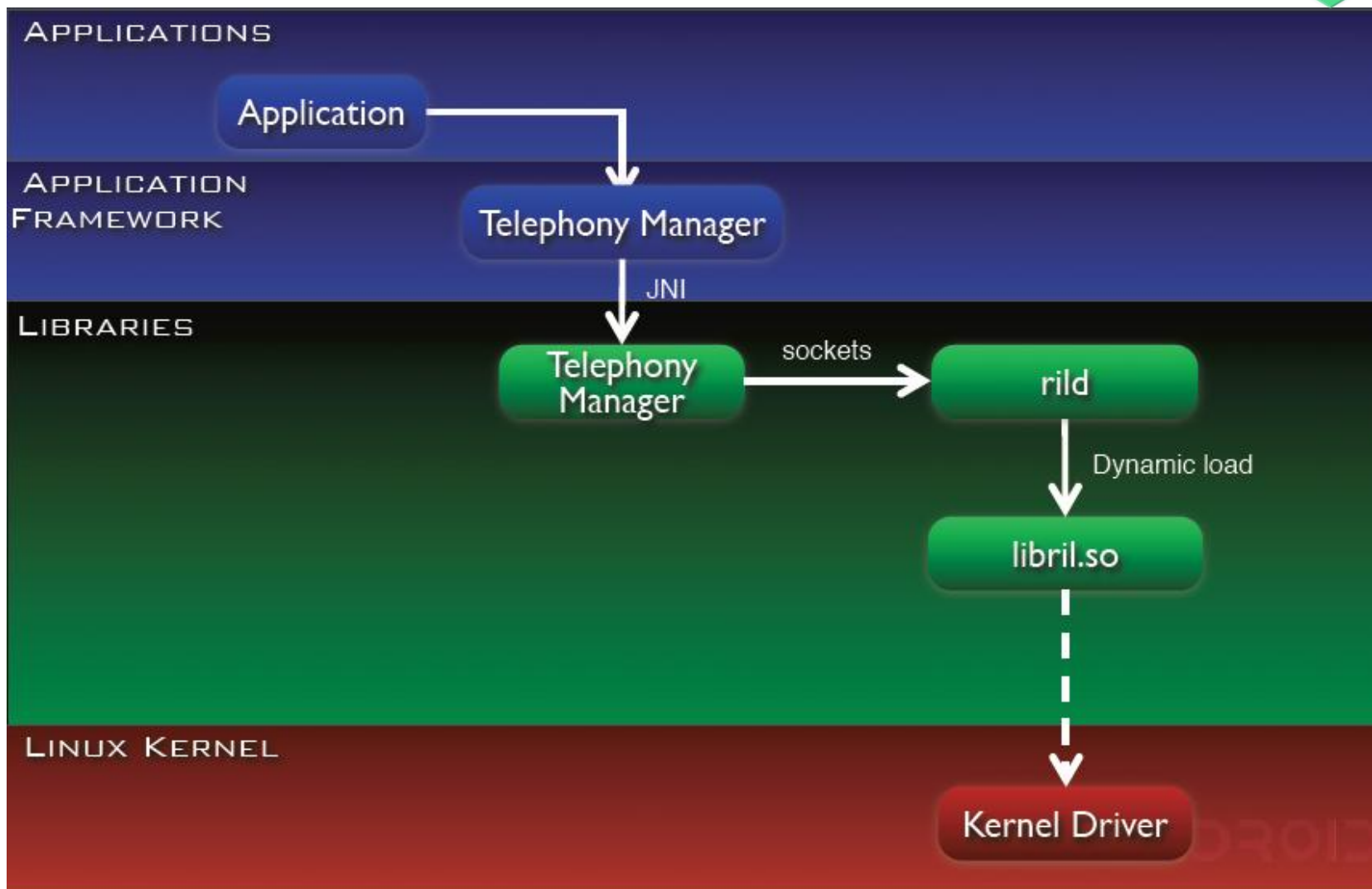
Interazione tra livelli 2/4



Interazione tra livelli 3/4



Interazione tra livelli 4/4





- **Concetti di Sistemi Operativi:** A. Silberschatz, P.B. Galvin, G. Gagne, Operating System Concepts, Wiley
- Android Open Source Project:
 - <https://source.android.com/>
 - <https://developer.android.com/docs>
 - <https://source.android.com/devices/tech/dalvik/index.html>
 - <https://developer.android.com/guide/platform>
- Anatomy & Physiology of an Android, Google I/O 2008